

Authentication & authorization

COMPLETE LEARNING GUIDE

What You'll Learn

Authentication answers 'who is this?' and authorization answers 'what are they allowed to do?' — two distinct problems that are easy to conflate but need to be reasoned about separately. This guide covers hashing passwords correctly (never storing them in plaintext or with a fast general-purpose hash), the trade-offs between session-based auth (server-tracked, cookie-referenced) and token-based auth (stateless JWTs), building role-based access control as reusable middleware, handling token expiry and refresh safely, and a brief grounding in OAuth2 for third-party login. Getting this layer wrong is one of the most common sources of real production security incidents, so the emphasis throughout is on the specific mistakes that turn a reasonable-looking auth implementation into an exploitable one.

1. Authentication vs. Authorization

Authentication (authn) verifies identity — logging in with a username/password, or presenting a valid token, proves you are who you claim to be. Authorization (authz) is a separate, subsequent question: given that verified identity, is this specific action on this specific resource allowed? A system can authenticate a user perfectly and still have a critical bug if it forgets to check authorization — for example, correctly verifying a JWT belongs to user 42, but then fetching /orders/99 without checking that order 99 actually belongs to user 42. Every protected route needs both checks, and they should be implemented as clearly separate steps, not tangled together.

2. Password Hashing

Passwords must never be stored in plaintext or reversibly encrypted — they must be hashed with a slow, purpose-built algorithm (bcrypt, scrypt, or Argon2), never a fast general-purpose hash like MD5 or SHA-256, which are designed for speed and make brute-forcing billions of guesses per second trivial on modern hardware. bcrypt automatically generates and stores a random salt per password, meaning two users with the same password get completely different hashes — this defeats precomputed rainbow-table attacks. The 'cost factor' (rounds) controls how slow the hash is to compute, which is a deliberate, tunable trade-off against brute-force attempts.

JAVASCRIPT

```
const bcrypt = require("bcrypt");

async function registerUser(email, plainPassword) {
  const passwordHash = await bcrypt.hash(plainPassword, 12); // cost factor 12
  return db.user.create({ data: { email, passwordHash } });
}

async function verifyLogin(email, plainPassword) {
  const user = await db.user.findUnique({ where: { email } });
  if (!user) return null;
  const valid = await bcrypt.compare(plainPassword, user.passwordHash);
  return valid ? user : null;
}
```

3. Session-Based Authentication

In session-based auth, the server creates a session record (in memory, a database, or Redis) after a successful login, and sends the client an opaque session ID inside a cookie. On each subsequent request, the browser automatically attaches that cookie, and the server looks up the session to identify the user. This gives the server full control — a session can be instantly invalidated server-side (immediate logout, forced revocation) — but requires shared session storage across every server instance in a horizontally scaled deployment, since any instance might need to look up any session.

4. Token-Based Authentication with JWT

A JSON Web Token packs claims (like user ID and role) into a signed, self-contained token that the server can verify without a database lookup, using a secret (HMAC) or public/private key pair (RSA/ECDSA). This statelessness makes JWTs attractive for horizontally scaled APIs and microservices — any server instance with the shared secret/public key can verify the token independently. The trade-off is that a JWT can't be individually revoked before its expiry without extra infrastructure (a denylist), since the server isn't tracking active tokens the way it tracks sessions — which is why short expiry times paired with a refresh-token flow are standard practice rather than issuing long-lived JWTs.

JAVASCRIPT

```
const jwt = require("jsonwebtoken");

function issueToken(user) {
  return jwt.sign(
    { sub: user.id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: "15m" }
  );
}

function requireAuth(req, res, next) {
  const authHeader = req.headers.authorization; // "Bearer <token>"
  const token = authHeader?.split(" ")[1];
  if (!token) return res.status(401).json({ error: "Missing token" });
}
```



```
try {  
  req.user = jwt.verify(token, process.env.JWT_SECRET);  
  next();  
} catch {  
  res.status(401).json({ error: "Invalid or expired token" });  
}  
}
```

5. Role-Based Access Control (RBAC) Middleware

Once a request is authenticated (req.user populated by the auth middleware above), authorization middleware can check the user's role or permissions against what the route requires, and reject the request before the route handler's own business logic even runs. This keeps 'can this user do this?' logic centralized and declarative on the route definition, rather than scattered as ad hoc if-checks inside every handler.

JAVASCRIPT

```
function requireRole(...allowedRoles) {
  return (req, res, next) => {
    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ error: "Insufficient permissions" });
    }
    next();
  };
}

app.delete(
  "/api/students/:id",
  requireAuth,
  requireRole("admin"),
  deleteStudentHandler
);
```

6. Refresh Tokens and Expiry

Short-lived access tokens (minutes, not days) limit the damage window if one is stolen, but forcing a full re-login every 15 minutes would be unusable. The standard pattern pairs a short-lived access token with a long-lived refresh token, stored more securely (an httpOnly cookie, or server-side) and used only to request a new access token when the old one expires — never sent with every ordinary API request. Refresh tokens should be revocable server-side (stored in a database, checked and invalidated on logout or suspected compromise) since, unlike access tokens, they live long enough that revocability actually matters.

7. OAuth2 and Third-Party Login

OAuth2 is a delegation protocol: it lets your app receive proof that a user authenticated with a trusted third party (Google, GitHub) without your app ever seeing that third party's password. The Authorization Code flow — the standard for a web app with a backend — redirects the user to the provider's login page, receives a short-lived authorization code back, and the app's backend exchanges that code (with a secret) for tokens, keeping the exchange off the client entirely. It's worth being explicit that OAuth2 by itself is about delegated authorization/authentication with a third party — your own app still needs its own session or token issued after that exchange completes to manage its own logged-in state.

Common Pitfalls

- Storing passwords in plaintext or hashed with a fast general-purpose algorithm (MD5/SHA-256) instead of bcrypt/Argon2.
- Storing a JWT in localStorage, which is readable by any JavaScript running on the page — making it a direct target for XSS-based token theft.
- Issuing long-lived access tokens with no expiry, removing any practical way to revoke access if a token is compromised.
- Trusting a client-supplied user ID or role in the request body instead of deriving identity solely from the verified token/session.
- Confusing authentication with authorization — verifying who someone is but forgetting to check whether they're allowed to access the specific resource requested.
- Not invalidating sessions or refresh tokens on logout, leaving them usable until natural expiry even after the user explicitly logged out.
- Using cookie-based sessions without CSRF protection, since browsers attach cookies automatically to cross-site requests unless explicitly guarded against.

Key Takeaways

- Hash passwords with bcrypt/Argon2, never store or reversibly encrypt them, and never use a fast general-purpose hash.
- Sessions are server-tracked and instantly revocable; JWTs are stateless and scale better but are hard to revoke before expiry.
- Authentication proves identity; authorization checks permission on a specific resource — both are required on every protected route.
- Pair short-lived access tokens with a revocable, longer-lived refresh token rather than issuing one long-lived token.
- Never trust client-supplied identity or role fields — always derive them from the verified token or session.
- OAuth2 delegates authentication to a trusted provider; your app still issues and manages its own session/token afterward.

Learn More

OWASP — Authentication Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

jwt.io — Introduction to JSON Web Tokens

<https://jwt.io/introduction>

Auth0 — OAuth 2.0 Authorization Code Flow

<https://auth0.com/docs/get-started/authentication-and-authorization-flow/authorization-code-flow>